

Core Concepts of Machine Learning

Unit 1: Problem Framing, Learning Tasks, Learning Scenarios & Data Preparation

Dr. Gopal Chandra Jana

Dept. of CSE, Sharda University
Machine Learning (CSE473) — Unit 1

July 22, 2026

Course Page:

<https://www.gcjana.in/courses/shardauniversity/2601/machine-learning/>

Outline

- 1 What is Machine Learning?
- 2 ML Applications and Standard Learning Tasks
- 3 Learning Stages, Scenarios and Generalization
- 4 Data Preparation and Feature Engineering
- 5 Summary and Practice

Learning Outcomes of Unit 1

After this unit, a student will be able to...

- 1 **Define** machine learning and identify problems suited to it.
- 2 **Frame** a real-world task as a well-posed ML problem.
- 3 **Distinguish** the standard learning tasks: classification, regression, ranking, clustering, dimensionality reduction.
- 4 **Explain** the learning stages — features, labels, hyperparameters, train/validation/test splits, loss functions.
- 5 **Compare** the learning scenarios and reason about **generalization**.
- 6 **Apply** data preprocessing: cleaning, integration, reduction, transformation and discretization — with calculations.

Mapping to the Syllabus

A. What is ML, problem framing, applications, standard tasks. **B.** Learning stages, scenarios, generalization. **C.** Data preparation, preprocessing, cleaning, integration, reduction, transformation.

What is Machine Learning?

Arthur Samuel (1959)

“The field of study that gives computers the ability to **learn without being explicitly programmed.**”

Tom Mitchell (1997) — the formal definition

A computer program is said to learn from **experience** E with respect to some class of **tasks** T and **performance measure** P , if its performance at tasks in T , as measured by P , **improves with experience** E .

Mohri et al. (2018)

ML consists of *computational methods using experience to improve performance or make accurate predictions.*
Here *experience* = past **data**.

Applying Mitchell's $\langle T, P, E \rangle$

Spam filter

T	classify e-mail as spam / not spam
P	% of e-mails correctly classified
E	a corpus of labelled e-mails

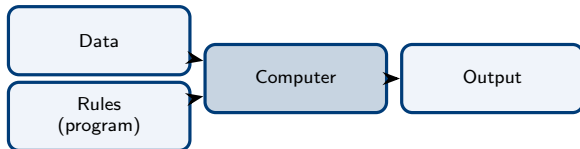
Chess program

T	playing chess
P	fraction of games won
E	playing games against itself

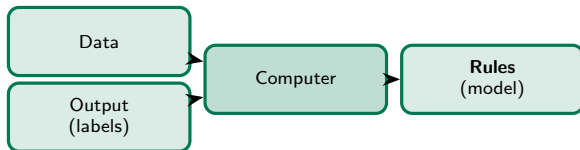
Try it: write $\langle T, P, E \rangle$ for a *handwriting recogniser*.

Traditional Programming vs. Machine Learning

Traditional Programming



Machine Learning



The flip: we no longer write the rules — we learn them from examples.

Why the flip matters

Nobody can write down the exact rule that separates a handwritten '2' from a '7', or a spam e-mail from a genuine one — but we can easily **collect examples** of each.

What Kinds of Problems Can ML Tackle?

ML shines when...

- The rules are **too complex** to hand-code (vision, speech).
- Rules **change over time** (spam, fraud, fashion).
- The problem needs **personalisation** at scale (recommendations).
- There is **plenty of data** but no clear formula.
- Human experts *can* do it but cannot **explain how** (tacit knowledge).
- We must find **hidden patterns** in large data.

ML is the wrong tool when...

- A **simple rule** already solves it ("is age ≥ 18 ?").
- You need **provable exactness** (payroll, cryptography).
- There is **no data**, or the data has no signal.
- Errors are **unacceptable** and unexplainable.
- The pattern does not **repeat** (one-off events).

Rule of thumb

If you can write the rule in a few lines, write the rule.
Use ML when the rule must be **discovered** from data.

The ML Mindset

Software engineering mindset

- **Deterministic**: same input → same, provably correct output.
- Correctness verified by **testing** against a specification.
- A bug is a **logic error** to be fixed.

ML mindset

- **Statistical**: predictions carry *uncertainty*.
- Judged by **metrics on unseen data**, not a spec.
- A “bug” may be **bad data** or a wrong metric.
- Progress is **empirical** — experiment, measure, iterate.

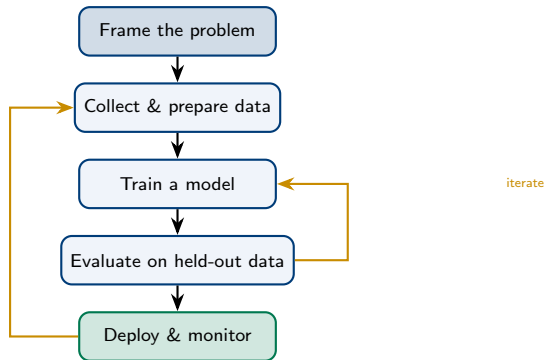


Figure: ML is an *empirical loop*, not a one-shot build.

Introduction to ML Problem Framing

Framing = turning a business/real problem into a learning problem

Ask, in order: (1) What outcome do we want? (2) What would we *predict*? (3) What data do we have? (4) How will we measure success?

Step-by-step recipe

- 1 State the **goal** in plain language.
- 2 Define the **ideal outcome** (business metric).
- 3 Choose the **output** (label) the model predicts.
- 4 Decide the **metric** (accuracy, RMSE, ...).
- 5 List candidate **features** available *at prediction time*.
- 6 Define a simple **baseline** to beat.

Worked framing — “reduce student dropout”

- **Goal**: fewer students drop out.
- **Ideal outcome**: counsellors reach at-risk students early.
- **Output**: $P(\text{dropout within a semester})$.
- **Metric**: recall at a fixed alert budget.
- **Features**: attendance, internal marks, fee status, LMS activity.
- **Baseline**: flag anyone with attendance $< 60\%$.

Common ML Problems (by output type)

Problem	Model predicts	Example
Binary classification	one of two classes	spam / not spam
Multi-class	one of K classes	digit 0–9; disease type
Multi-label	any subset of labels	tags on a photo
Regression	a real number	house price; temperature
Ranking	an ordering of items	search results; recommendations
Clustering	groups (no labels given)	customer segments
Anomaly detection	normal / unusual	fraud, machine failure
Sequence / generation	structured output	translation, captioning

Quick engagement — name the problem type

(i) Rainfall in mm $\rightarrow$? (ii) Grouping news articles, no labels $\rightarrow$? (iii) Ordering products in a feed $\rightarrow$? [(i) regression, (ii) clustering, (iii) ranking]

ML Use Cases Around Us

Everyday

- E-mail spam filtering, smart reply
- Face unlock, photo tagging
- Voice assistants, live captions
- Maps: ETA prediction, route choice
- Recommendations: video, music, shopping

Industry & science

- Fraud & credit-risk scoring
- Predictive maintenance in factories
- Drug discovery, protein structure
- Demand forecasting, price optimisation

Campus-scale examples to think about

- Predicting library book demand before exams.
- Timetable conflict detection.
- Detecting plagiarism across submissions.
- Estimating mess food quantity to cut waste.

Discussion prompt

For *one* of the above: what is the **label**, what **features** exist *before* prediction time, and what could go wrong if the model is confidently wrong?

Identifying Good Problems for ML

A good ML problem has...

- 1 A **clear, measurable** objective.
- 2 **Abundant, relevant** data — and labels if supervised.
- 3 A pattern that is **learnable** and **stable** over time.
- 4 Tolerance for **occasional errors**.
- 5 Features available **at prediction time** (no leakage).
- 6 Clear **action** taken from the prediction.

Data leakage — a classic trap

Using a feature that is only known *after* the label exists.

Example: predicting whether a patient has a disease, using the field “treatment prescribed”. Accuracy looks superb in testing and collapses in production.

Always ask

“Would I actually have this column, with this value, at the moment I need the prediction?” If no $\Rightarrow$ drop it.

Hard ML Problems

What makes a problem hard

- **Scarce or costly labels** (expert annotation).
- **Class imbalance** — e.g. 0.1% fraud.
- **Concept drift** — the world changes after training.
- **Noisy or biased** data.
- **Causality** needed, not mere correlation.
- **Explainability** demanded (law, medicine).
- Very **high-dimensional** data (curse of dimensionality).

The imbalance trap — a number to remember

If only 1 in 1000 transactions is fraud, a model that predicts “*never fraud*” achieves

$$\text{accuracy} = \frac{999}{1000} = \mathbf{99.9\%}$$

... and catches **zero** fraud.

⇒ accuracy is the *wrong metric*; use precision, recall, F1, or AUC.

Lesson

Choosing the **metric** is part of framing the problem — not an afterthought.

Application 1 — Image Recognition

The task

Map an image (a grid of pixel intensities) to a label or a set of objects.

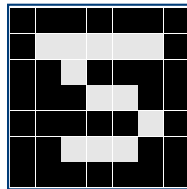
Input: $28 \times 28 = 784$ pixels (MNIST) or millions for photographs.

Why ML and not rules

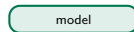
A '2' can be thin, slanted, looped or broken — infinitely many shapes. No hand-written rule covers them; a model **learns** the pattern from thousands of labelled examples.

Uses

handwriting/OCR and pin-code sorting, face verification, medical imaging, self-driving perception, quality inspection.



pixels $\rightarrow$ features



$P(2) = 0.9$
 $P(7) = 0.0$
 $P(3) = 0.0$

Figure: Pixels in, *confidence over classes* out.

Application 2 — Speech Recognition

The task

Convert an acoustic waveform into text: **speech-to-text** (ASR).

audio signal $\longrightarrow$ word sequence

Why it is hard

- Huge **speaker variation**: accent, pitch, speed, emotion.
- **Background noise** and overlapping speakers.
- **Homophones**: “to / too / two”; context decides.
- Continuous speech has **no clear word boundaries**.

Related tasks

speaker identification, language identification, wake-word detection, emotion recognition.

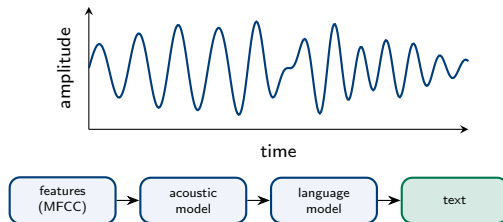


Figure: The classical ASR pipeline; modern systems learn it end-to-end.

Application 3 — Medical Diagnosis

The task

From patient data (symptoms, vitals, lab results, scans) predict a **diagnosis**, a **risk score**, or a **prognosis**.

Examples

- Tumour detection in MRI / CT.
- Diabetic-retinopathy screening from retinal photos.
- ECG arrhythmia classification.
- Sepsis early-warning from ICU vitals.

Special demands in healthcare

- **Errors cost lives** $\Rightarrow$ high recall on serious conditions.
- **Explainability** required for clinician trust and regulation.
- **Bias**: a model trained on one population may fail on another.
- **Privacy**: patient data is sensitive and regulated.
- ML **assists**, it does not replace, the clinician.

Metric note

For screening we usually prefer **high recall** (miss few sick patients) and accept more false alarms, which a second test can filter.

Application 4 — Statistical Arbitrage

The idea

In finance, **statistical arbitrage** uses ML on large volumes of market data to find short-lived **mispricings** between related securities and trade them systematically.

Typical formulation

- **Pairs trading**: two historically correlated stocks diverge $\Rightarrow$ buy the laggard, sell the leader, profit when the spread reverts.
- Cast as **regression** (predict the spread) or **classification** (will the spread narrow?).
- Features: price history, volume, volatility, sector, news sentiment.

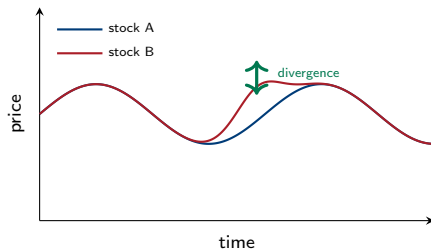


Figure: A temporary divergence between correlated assets is the trading signal.

Caution

Markets are noisy and *non-stationary* — a model that worked last year may fail this year (**concept drift**).

Application 5 — Learning Associations

Association rule learning

Discover rules of the form $X \Rightarrow Y$ (“customers who buy X also buy Y ”) from transaction data — **market-basket analysis**.

Three standard measures

$$\text{support}(X \Rightarrow Y) = P(X \cup Y)$$

$$\text{confidence}(X \Rightarrow Y) = P(Y \mid X) = \frac{P(X \cup Y)}{P(X)}$$

$$\text{lift}(X \Rightarrow Y) = \frac{P(Y \mid X)}{P(Y)}$$

Lift $> 1 \Rightarrow$ positive association.

Numeric example

Out of **1000** transactions: bread appears in 400, butter in 300, and *both* in 240.

$$\text{support} = \frac{240}{1000} = \mathbf{0.24}$$

$$\text{confidence} = \frac{240}{400} = \mathbf{0.60}$$

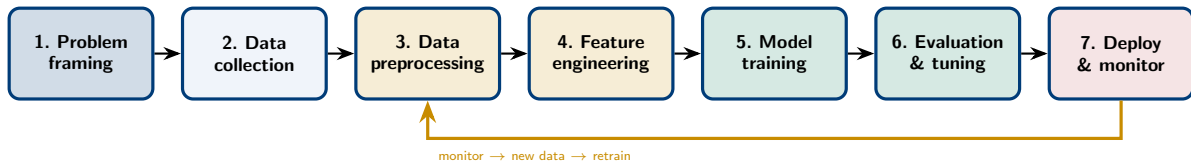
$$\text{lift} = \frac{0.60}{300/1000} = \frac{0.60}{0.30} = \mathbf{2.0}$$

Lift = 2: buying bread makes butter **twice as likely** as baseline.

Uses

store layout, cross-selling, bundle offers, web-page prefetching.

The Machine Learning Pipeline



Where the time actually goes

Practitioners commonly report that **60–80%** of project effort is spent on steps 2–4 (data), not on the algorithm.

Golden rule

Garbage in, garbage out. A simple model on clean, well-engineered features usually beats a complex model on poor data — which is why Part C of this unit is about **data**.

Standard Task 1 — Classification

Definition

Learn $h : \mathcal{X} \rightarrow \mathcal{Y}$ where $\mathcal{Y}$ is a **finite, discrete** set of categories.

- Binary: $\mathcal{Y} = \{-1, +1\}$ or $\{0, 1\}$
- Multi-class: $|\mathcal{Y}| = K > 2$

Examples

spam detection, digit recognition, disease vs. healthy, sentiment (pos/neg/neutral).

Typical algorithms

logistic regression, k -NN, decision trees, SVM, naive Bayes, neural networks.

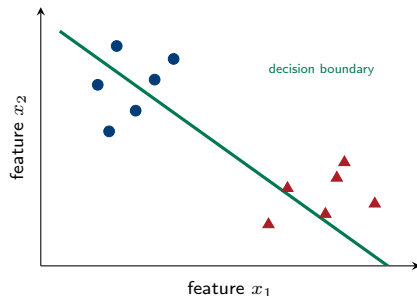


Figure: Classification learns a *decision boundary* separating the classes.

Standard Task 2 — Regression

Definition

Learn $h : \mathcal{X} \rightarrow \mathbb{R}$: the output is a **continuous** value.
Quality is measured by the *magnitude* of the error, e.g.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Examples

house price, tomorrow's temperature, delivery time, electricity demand, marks from study hours.

Classification vs. regression

"Which class?" vs. *"How much?"*

Predicting rain *yes/no* is classification; predicting *how many mm* is regression.

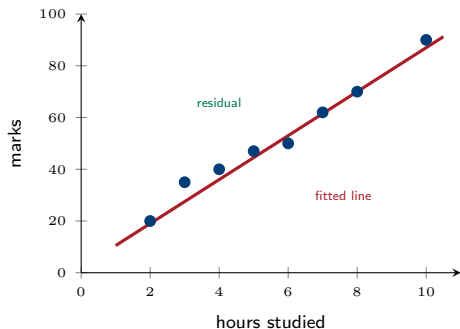


Figure: Regression fits a function; residuals are the prediction errors.

Standard Task 3 — Ranking

Definition

Learn to **order** a set of items by relevance to a query or user, rather than to label each item independently.

$$h : (\text{query}, \text{items}) \longrightarrow \text{permutation}$$

Examples

web search results, product recommendations, news feeds, resume shortlisting, answer ranking in Q&A sites.

Why it is its own task

Only the **relative order** matters — getting the top few right matters far more than the tail. Metrics: **NDCG**, **MAP**, $\text{precision}@k$.

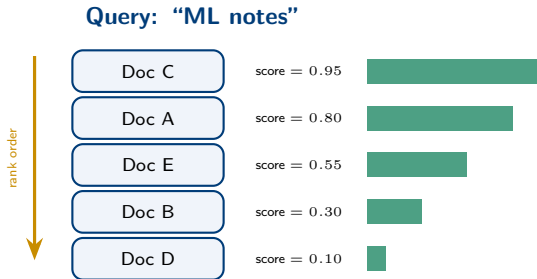


Figure: The model scores items, then sorts them; the *order* is the output.

Standard Task 4 — Clustering

Definition

Partition **unlabelled** data into groups so that points in a group are similar to each other and dissimilar to other groups.

Key points

- **Unsupervised** — no ground-truth labels.
- Needs a **similarity/distance** measure.
- “Correct” answer is often subjective.
- Algorithms: **k-means**, hierarchical, DBSCAN, GMM.

Uses

customer segmentation, topic grouping, image compression, anomaly detection, gene-expression analysis.

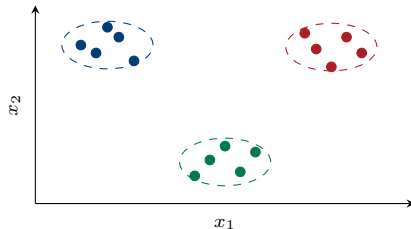


Figure: Structure is discovered without any labels being provided.

Standard Task 5 — Dimensionality Reduction

Definition

Map high-dimensional data to a lower-dimensional representation while **preserving the essential structure**:

$$\mathbb{R}^D \longrightarrow \mathbb{R}^d, \quad d \ll D.$$

Why

- Beat the **curse of dimensionality**.
- Remove noise and redundancy.
- Enable **visualisation** (2-D / 3-D).
- Speed up training, reduce storage.

Methods

PCA (linear), **manifold learning**: Isomap, LLE, t-SNE, UMAP; autoencoders.

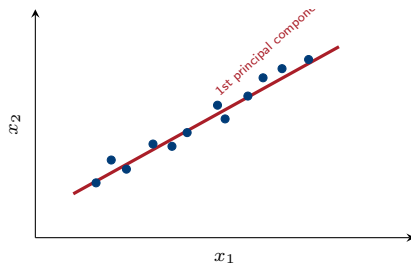


Figure: Data in 2-D lies close to a 1-D direction; projecting onto it keeps most of the variance.

Quick Check — Round 1 (Concepts & Tasks)

[Q1]

Write Mitchell's definition and give $\langle T, P, E \rangle$ for a chess-playing program.

[Q2]

A bank's fraud data has 0.1% positives. Why is accuracy a poor metric here?

[Q3]

Classify each as classification, regression, ranking, clustering or dimensionality reduction:

(i) predicting rainfall in mm (ii) grouping customers with no labels (iii) ordering search results (iv) compressing 784 pixels into 2 numbers for a plot.

[Q4]

In 1000 transactions, milk appears 500 times, cereal 200 times, both 150 times. Compute support, confidence and lift for milk $\Rightarrow$ cereal.

Answers — Round 1

[A1]

A program *learns* from experience E w.r.t. tasks T and measure P if its performance at T , measured by P , improves with E . Chess: T = playing chess, P = fraction of games won, E = games played (e.g. self-play).

[A2]

Predicting “never fraud” scores **99.9%** accuracy while catching **zero** fraud. With extreme imbalance, use **precision, recall, F1 or AUC** instead.

[A3]

(i) **regression** (ii) **clustering** (iii) **ranking** (iv) **dimensionality reduction**.

[A4]

support = $\frac{150}{1000} = \mathbf{0.15}$; confidence = $\frac{150}{500} = \mathbf{0.30}$; lift = $\frac{0.30}{200/1000} = \frac{0.30}{0.20} = \mathbf{1.5}$ (positive association).

Learning Stages — The Vocabulary (Mohri et al.)

Term	Meaning
Examples / instances	The items of data used for learning or evaluation.
Features	The attribute vector $\mathbf{x}$ describing an example.
Labels	The value/category y attached to an example (supervised tasks).
Training sample	Examples used to <i>fit</i> the model.
Validation sample	Examples used to <i>tune hyperparameters</i> / select a model.
Test sample	Examples used <i>once</i> to report final performance.
Hyperparameters	Settings <i>not</i> learned from data (e.g. k , depth, learning rate).
Hypothesis set	The family $\mathcal{H}$ of candidate functions the learner may pick from.
Loss function	$L(y, \hat{y})$: the penalty for predicting $\hat{y}$ when the truth is y .

One-line story

A learning algorithm searches a **hypothesis set** $\mathcal{H}$ for the hypothesis that minimises the **loss** on the **training sample**, using the **validation sample** to fix hyperparameters, and is judged on the **test sample**.

Features and Labels — A Concrete Table

Predicting whether a student passes

Student	Features $\mathbf{x}$			Label y
	Attendance (%)	Internal marks	Hours/week	Passed?
s_1	92	27	10	Yes
s_2	61	14	3	No
s_3	78	22	6	Yes
s_4	45	11	2	No

Notation

- One row = one **example** $(\mathbf{x}_i, y_i)$.
- $\mathbf{x}_1 = (92, 27, 10)$, $y_1 = \text{Yes}$.
- Training set $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$.

Feature engineering preview

We could *add* derived features: marks per hour studied, attendance trend, or a binary “attendance $\geq 75\%$ ”.

Good features often matter more than the algorithm.

Training, Validation and Test Samples



The golden rule

Never train or tune on the test set. Each time you peek at test data to make a decision, it becomes a little more like validation data — and your reported number becomes optimistic.

k -fold cross-validation

When data is scarce: split into k folds, train on $k - 1$ and validate on the remaining one, rotating k times, then average. Common choice $k = 5$ or 10 .

Parameters vs. Hyperparameters

Parameters — *learned*

Values the algorithm fits **from the training data**.

- Weights w and bias b in linear/logistic regression
- Split thresholds in a decision tree
- Connection weights in a neural network

Hyperparameters — *chosen*

Settings fixed **before** training, tuned on the **validation** set.

- k in k -NN and in k -means
- Tree depth, minimum samples per leaf
- Learning rate, batch size, epochs
- Regularisation strength λ

How hyperparameters are chosen

- **Grid search** — try all combinations on a grid.
- **Random search** — sample combinations (often better per unit time).
- **Bayesian optimisation** — model the response surface.
- **Evolutionary search** — GA/PSO over configurations.

Memory hook

*Parameters are learned by the model; hyperparameters are learned by **you** (or by an outer search loop).*

Loss Functions

Definition

A **loss** $L(y, \hat{y}) \geq 0$ measures the penalty of predicting $\hat{y}$ when the truth is y . Learning minimises the **empirical risk**

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m L(y_i, h(\mathbf{x}_i)).$$

Common losses

- **0–1 loss** (classification): $L = \mathbb{I}[y \neq \hat{y}]$
- **Squared** (regression): $L = (y - \hat{y})^2$
- **Absolute**: $L = |y - \hat{y}|$
- **Hinge** (SVM): $L = \max(0, 1 - y\hat{y})$
- **Cross-entropy**: $L = -\sum_k y_k \log \hat{y}_k$

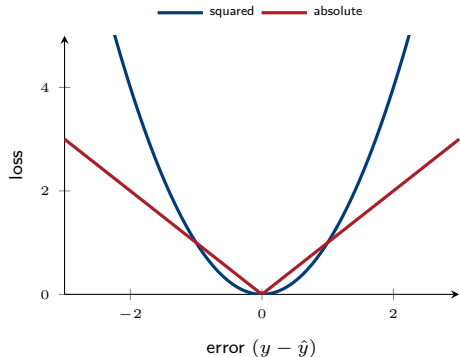


Figure: Squared loss punishes large errors much harder, so it is more sensitive to *outliers* than absolute loss.

Numerical Example — Computing the Loss

Regression: four predictions

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	
True y_i	3.0	5.0	7.0	9.0	
Predicted $\hat{y}_i$	2.5	5.5	6.0	9.5	
Error $e_i = y_i - \hat{y}_i$	+0.5	-0.5	+1.0	-0.5	
e_i^2	0.25	0.25	1.00	0.25	$\Sigma = 1.75$

Regression metrics

$$\text{MSE} = \frac{1.75}{4} = \mathbf{0.4375}$$

$$\text{RMSE} = \sqrt{0.4375} = \mathbf{0.6614}$$

$$\text{MAE} = \frac{0.5+0.5+1.0+0.5}{4} = \mathbf{0.625}$$

Classification: 0–1 loss

On 10 test samples the model gets **2 wrong**:

$$\hat{R} = \frac{2}{10} = \mathbf{0.2}, \quad \text{accuracy} = 1 - 0.2 = \mathbf{80\%}$$

Note RMSE is back in the *original units* of y , which is why it is often reported instead of MSE.

Hypothesis Set and Hypothesis Testing

Hypothesis set $\mathcal{H}$

The family of functions the learner is allowed to choose from — e.g. all linear functions, all trees of depth ≤ 5 , all networks of a fixed architecture.

- $\mathcal{H}$ too **small** $\Rightarrow$ underfitting (cannot express the truth).
- $\mathcal{H}$ too **large** $\Rightarrow$ overfitting (can fit noise).

Complexity measures

VC-dimension, Rademacher complexity — formal ways to say “how rich is $\mathcal{H}$ ”, used to bound generalization error.

Hypothesis testing (statistics)

Used to decide whether an observed improvement is **real** or chance.

- H_0 : the two models perform equally.
- H_1 : model B is better.
- Compute a test statistic and a **p-value**; reject H_0 if $p < \alpha$ (say 0.05).
- Typical tests: paired t -test over CV folds, McNemar's test for classifiers.

Why it matters

“91.2% vs. 91.0%” on one split may be pure noise.
Report variation across folds/seeds, not a single number.

Learning Scenarios — Overview

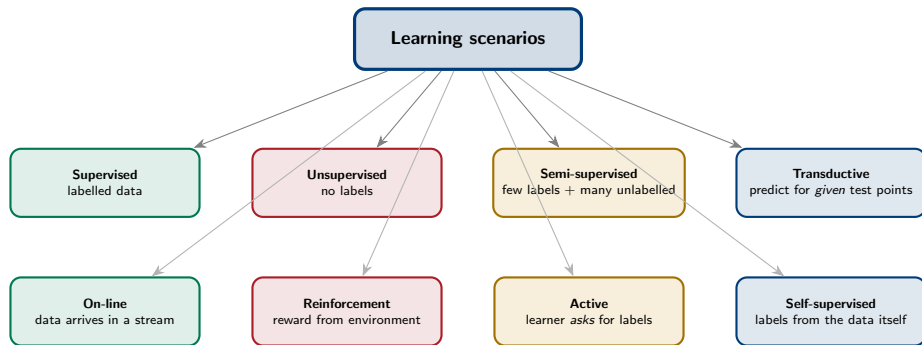


Figure: The scenarios differ in *what information* the learner receives and *when* it receives it.

Supervised and Unsupervised Learning

Supervised learning

The learner receives **labelled** examples $(\mathbf{x}_i, y_i)$ and predicts labels for unseen points.

- Tasks: classification, regression, ranking.
- Needs: a labelled training set (often costly).
- Success: low error on *new* data.

Example: predict pass/fail from attendance and marks.

Unsupervised learning

The learner receives **unlabelled** data only and must find structure.

- Tasks: clustering, dimensionality reduction, density estimation, association rules.
- Needs: a notion of similarity.
- Success: harder to define; often judged by usefulness.

Example: segment customers into behaviour groups.

Key contrast

Supervised learning is told *the answer* during training; unsupervised learning must **invent the question and the answer** from structure alone.

Semi-Supervised Learning and Transductive Inference

Semi-supervised learning

Training uses a **small labelled** set plus a **large unlabelled** set.

- Motivation: labels are expensive (doctors, lawyers, annotators); raw data is cheap.
- Assumption: unlabelled data reveals the *shape* of the distribution (clusters, manifolds), guiding the boundary.
- Goal: predict well on **any** future point (inductive).

Transductive inference

The learner is given labelled data *and* the **specific unlabelled test points**, and only needs to label **those points** — not to produce a general rule.

- Easier problem: no need to generalise everywhere.
- Typical in graph-based label propagation.

Contrast in one line

Inductive: learn a rule for *all* future points.

Transductive: label *these* points.

On-line Learning and Active Learning

On-line learning

Data arrives **sequentially**; the model updates after each example. Training and testing are interleaved.

- Suits streams: click prediction, stock ticks, sensors.
- Handles **concept drift** naturally.
- Performance measured by cumulative **regret** or mistake count, not a fixed test set.

Contrast

Batch: see all data, then learn. **On-line**: learn as data flows.

Active learning

The learner **chooses which examples to have labelled**, querying an oracle (a human expert) for the most informative points.

- Goal: reach a target accuracy with **far fewer labels**.
- Query strategies: most uncertain point, largest disagreement in a committee, expected model change.

Where it pays

Medical images, legal documents — anywhere a label costs an expert's time.

Reinforcement Learning

Setting

An **agent** interacts with an **environment**: it observes a state, takes an action, receives a **reward**, and moves to a new state. The goal is to maximise **cumulative reward**.

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

where π is the policy and $\gamma \in [0, 1]$ the discount factor.

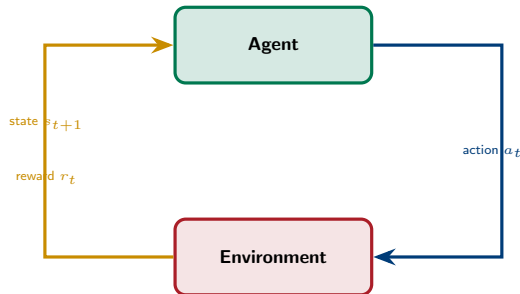


Figure: The agent–environment interaction loop.

What makes RL distinctive

No labelled examples — only **evaluative feedback**, often **delayed**. The agent must balance **exploration** and **exploitation**.

Examples

game playing (Chess, Go), robot control, traffic-signal control, recommendation with long-term engagement.

Learning Scenarios — Comparison Table

Scenario	Data given	Feedback	Typical use
Supervised	labelled $(\mathbf{x}, y)$	correct answer	spam, diagnosis
Unsupervised	$\mathbf{x}$ only	none	segmentation, PCA
Semi-supervised	few labels + many $\mathbf{x}$	partial	text/image with scarce labels
Transductive	labels + the actual test points	correct answer	graph label propagation
On-line	a stream of examples	per-step	click/stock prediction
Reinforcement	interaction traces	delayed reward	control, games
Active	learner queries an oracle	on demand	costly annotation

How to decide

Ask: *Do I have labels?* → *Are they enough?* → *Does data arrive at once or as a stream?* → *Do my actions change the world?* The answers point to the scenario.

Generalization — The Central Goal

Definition

Generalization is the ability to perform well on **unseen** data drawn from the same distribution. We minimise training error but *care about*

$$R(h) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [L(y, h(\mathbf{x}))].$$

The gap $R(h) - \hat{R}(h)$ is the **generalization gap**.

Overfitting vs. underfitting

- **Underfit**: high training *and* test error — model too simple.
- **Overfit**: low training, high test error — memorised noise.
- **Good fit**: both low, and close together.

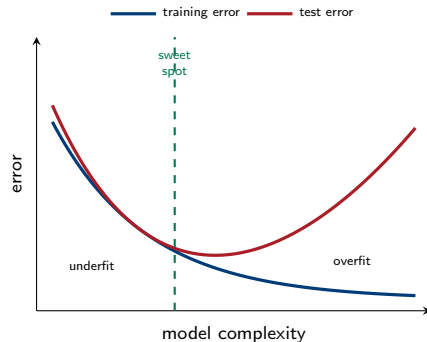


Figure: Test error is U-shaped in model complexity.

Bias–Variance Trade-off and Remedies

Decomposition (squared loss)

$$\mathbb{E}[(y - \hat{h})^2] = \underbrace{\text{Bias}^2}_{\text{wrong assumptions}} + \underbrace{\text{Variance}}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

- **High bias** → underfitting (too simple).
- **High variance** → overfitting (too flexible).

Intuition

Bias = consistently missing the target in the same direction. Variance = scattering widely around it depending on the training sample.

Remedies for overfitting

- More **training data**; data augmentation.
- **Regularisation** (L_1 , L_2 , dropout).
- **Simplify** the model / prune the tree.
- **Early stopping** using a validation curve.
- **Cross-validation** for honest selection.
- **Ensembles** (bagging reduces variance).

Remedies for underfitting

richer model, better/more features, less regularisation, train longer.

Quick Check — Round 2 (Stages, Scenarios, Generalization)

[Q1]

Distinguish a *parameter* from a *hyperparameter*; give one example of each.

[Q2]

Why must the test set never be used for tuning?

[Q3]

A model scores 99% on training data and 62% on test data. Name the problem and give three remedies.

[Q4]

Match: (i) labels for the given test points only, (ii) learner queries an oracle, (iii) reward-based interaction, (iv) data arrives as a stream
with: (a) active, (b) on-line, (c) transductive, (d) reinforcement.

Answers — Round 2

[A1]

A **parameter** is fitted *from* the data (e.g. regression weights w); a **hyperparameter** is set *before* training and tuned on validation data (e.g. k in k -NN, tree depth, learning rate).

[A2]

Because the test set is our only **unbiased** estimate of performance on unseen data. Tuning on it leaks information and produces an **optimistic**, unreliable number.

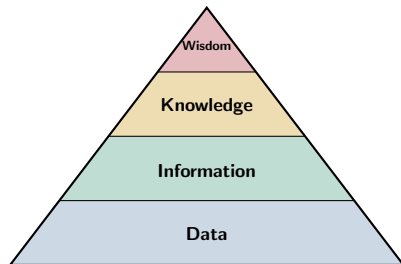
[A3]

Overfitting (huge generalization gap). Remedies: more data / augmentation, regularisation (L_1 , L_2 , dropout), a simpler model, early stopping, cross-validation, ensembling (any three).

[A4]

(i)–(c) transductive (ii)–(a) active (iii)–(d) reinforcement (iv)–(b) on-line.

Data, Information, Knowledge



The hierarchy

- **Data**: raw, unorganised facts. *"38.5"*
- **Information**: data + context. *"Patient's temperature is 38.5° C"*
- **Knowledge**: patterns/rules from information. *"Above 37.5° C indicates fever"*
- **Wisdom**: judgement in applying knowledge. *"Given the other symptoms, order this test"*

Where ML sits

Machine learning is the machinery that turns **data** into **knowledge** — automatically discovering the patterns that a human would otherwise have to write down.

Data Types and Attribute Scales

Type	Meaning	Examples	Valid operations
Nominal	names/categories, no order	gender, city, blood group	=, $\neq$, mode
Binary	nominal with 2 states	pass/fail, spam/ham	=, $\neq$
Ordinal	ordered, gaps not meaningful	grade (A,B,C), rating	<, >, median
Interval	ordered, equal gaps, no true zero	temperature ($^{\circ}\text{C}$), dates	+, -, mean
Ratio	interval with a true zero	height, weight, income, count	all, incl. $\times$, $\div$

Why the scale matters

20 $^{\circ}\text{C}$ is **not** “twice as hot” as 10 $^{\circ}\text{C}$ (interval, no true zero) — but Rs.20 is twice Rs.10 (ratio). Using the wrong operation silently corrupts your features.

Also distinguish

Discrete vs. **continuous**; **structured** (tables) vs. **unstructured** (text, image, audio) vs. **semi-structured** (JSON, XML).

Big Data — The Five V's

Volume
scale of
data

Velocity
speed of
arrival

Variety
text, image,
log, sensor

Veracity
noise &
trust

Value
usefulness
extracted

Scale intuition

kilobyte (KB)	10^3 bytes
megabyte (MB)	10^6
gigabyte (GB)	10^9
terabyte (TB)	10^{12}
petabyte (PB)	10^{15}
exabyte (EB)	10^{18}

Consequence for ML

Big data enables richer models, but forces us to care about **sampling**, **distributed storage/compute**, **streaming** algorithms, and above all **data quality** — more data is useless if it is wrong.

Reminder

“*Veracity*” is the V students most often forget — and the one that sinks the most projects.

Why Preprocess? Data Quality Dimensions

Quality dimensions (Han & Kamber)

- **Accuracy** — are the values correct?
- **Completeness** — are values missing?
- **Consistency** — do values contradict?
- **Timeliness** — is the data up to date?
- **Believability** — do users trust it?
- **Interpretability** — is it understandable?

Reality check

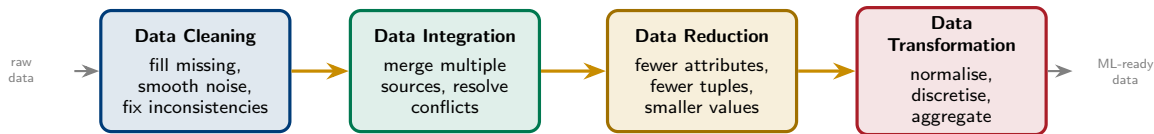
Real data is **dirty**: typos, duplicates, missing fields, inconsistent units (kg vs. lb), impossible values (age = -3), stale records.

Spot the problems

ID	Age	City	Income
1	21	Delhi	40000
2	-3	delhi	45000
3	22	Noida	-
4	220	NOIDA	50000
4	22	Noida	50000

Issues: impossible ages (-3, 220), missing income, inconsistent city casing, duplicate ID 4.

Major Tasks in Data Preprocessing



These are the four pillars of Part C

We now take each in turn, with worked calculations. Note the order is a guideline, not a law — in practice these steps interleave and repeat.

Data Cleaning — Missing Values

Standard strategies

- 1 **Ignore the tuple** — simple, but wasteful; fine only if the missing rate is tiny.
- 2 **Fill manually** — accurate, infeasible at scale.
- 3 **Global constant** — e.g. “Unknown”; can create a bogus category.
- 4 **Attribute mean/median** — easy; shrinks variance.
- 5 **Class-conditional mean** — mean of the same class; better.
- 6 **Predict it** — regression, k -NN, decision tree, EM.
Most accurate, most work.

Worked — mean imputation

Incomes (in thousands): 40, 45, ?, 55, 60.

$$\text{mean} = \frac{40 + 45 + 55 + 60}{4} = \frac{200}{4} = 50$$

Impute 50. Median here is also 50; with an outlier (say 400) the median would be the **safer** choice.

Think before you fill

Why is it missing? “Income blank” may itself signal something (a pattern called **missing not at random**). Adding a binary “was_missing” flag often helps.

Data Cleaning — Noisy Data and Binning

Noise = random error or variance in a measured variable

Handled by **binning** (smooth locally), **regression** (fit a function), **outlier analysis** (clustering), or combined human-machine inspection.

Sorted price data (9 values)

4, 8, 15, 21, 21, 24, 25, 28, 34

Equal-frequency partition into bins of 3:

- Bin 1: 4, 8, 15
- Bin 2: 21, 21, 24
- Bin 3: 25, 28, 34

Smoothing by bin *means*

means: $\frac{4+8+15}{3} = 9$, $\frac{21+21+24}{3} = 22$, $\frac{25+28+34}{3} = 29$

- Bin 1 $\rightarrow$ 9, 9, 9
- Bin 2 $\rightarrow$ 22, 22, 22
- Bin 3 $\rightarrow$ 29, 29, 29

Smoothing by bin *boundaries*

each value $\rightarrow$ the nearer of {min, max} of its bin:

- Bin 1 (4, 15) $\rightarrow$ 4, 4, 15
- Bin 2 (21, 24) $\rightarrow$ 21, 21, 24
- Bin 3 (25, 34) $\rightarrow$ 25, 25, 34

Data Cleaning as a Process

Step 1 — Discrepancy detection

Find what is wrong, using:

- **Metadata**: expected domain, range, type, dependencies.
- **Unique / consecutive / null rules**.
- **Field overloading** — two meanings packed into one column.
- Tools: **data scrubbing** (domain knowledge, e.g. postal-code checks), **data auditing** (find rules & outliers statistically).

Step 2 — Transformation

Fix what was found, using **data migration** tools and ETL scripts (e.g. replace “gender” → “sex”, standardise date formats).

It is iterative

The two steps **alternate**: fixing one problem exposes another. Modern practice favours **interactive** tools with immediate feedback, and **logging** every change for reproducibility.

Good habit

Never overwrite raw data. Keep raw/ immutable and write cleaned output to processed/, with a script that reproduces it.

Merging data from multiple sources

Combines databases, files, cubes into a coherent store.
Four classic problems:

- 1 **Entity identification problem**
- 2 **Redundancy and correlation**
- 3 **Tuple duplication**
- 4 **Data value conflicts**

1. Entity identification

Is `cust_id` in DB-A the same as `customer_number` in DB-B? Resolved using **metadata**: name, meaning, data type, allowed range, null rules.

3. Tuple duplication

The same real-world entity stored twice (often with slight differences), e.g. denormalised tables. Causes double counting and biased models.

4. Data value conflicts

The same attribute differs across sources because of:

- **Units**: kg vs. pounds; Rs. vs. \$.
- **Scales**: marks out of 100 vs. out of 10; grading systems.
- **Representation**: “M/F” vs. “Male/Female”; date formats.

Resolution needs explicit **schema and unit standardisation**.

Redundancy & Correlation — Nominal Data (χ^2 Test)

The χ^2 statistic

For a contingency table, with o_{ij} observed and e_{ij} expected counts:

$$\chi^2 = \sum_i \sum_j \frac{(o_{ij} - e_{ij})^2}{e_{ij}}, \quad e_{ij} = \frac{\text{count}(A = a_i) \times \text{count}(B = b_j)}{n}$$

A large $\chi^2 \Rightarrow$ reject independence $\Rightarrow$ the attributes are **correlated** (one may be redundant).

Observed ($n = 1500$)

	male	female	total
fiction	250	200	450
non-fiction	50	1000	1050
total	300	1200	1500

Expected counts

$$e_{11} = \frac{450 \times 300}{1500} = 90, \quad e_{12} = \frac{450 \times 1200}{1500} = 360, \\ e_{21} = \frac{1050 \times 300}{1500} = 210, \quad e_{22} = \frac{1050 \times 1200}{1500} = 840.$$

The full computation follows on the next slide.

Numerical Example — Computing χ^2

Cell-by-cell contributions

Cell	Observed o	Expected e	$(o - e)^2/e$
(fiction, male)	250	90	$\frac{(250-90)^2}{90} = \frac{25600}{90} = 284.44$
(fiction, female)	200	360	$\frac{(200-360)^2}{360} = \frac{25600}{360} = 71.11$
(non-fiction, male)	50	210	$\frac{(50-210)^2}{210} = \frac{25600}{210} = 121.90$
(non-fiction, female)	1000	840	$\frac{(1000-840)^2}{840} = \frac{25600}{840} = 30.48$
Total χ^2			507.93

Decision

Degrees of freedom = $(2 - 1)(2 - 1) = 1$.

Critical value at $\alpha = 0.001$ is 10.828.

Since $507.93 \gg 10.828$, **reject independence**.

Interpretation

Gender and preferred reading are **strongly correlated** in this data. For integration, one attribute may be partly **redundant** given the other.

Correlation & Covariance — Numeric Data

Pearson correlation coefficient

$$r_{A,B} = \frac{\sum_{i=1}^n (a_i - \bar{A})(b_i - \bar{B})}{n \sigma_A \sigma_B} = \frac{\sum a_i b_i - n \bar{A} \bar{B}}{n \sigma_A \sigma_B}, \quad \text{Cov}(A, B) = \frac{\sum a_i b_i}{n} - \bar{A} \bar{B}$$

$r > 0$ positive, $r = 0$ uncorrelated, $r < 0$ negative; $-1 \leq r \leq 1$.

Data: hours studied vs. marks

A (hours)	2	4	6	8	10
B (marks)	20	40	50	70	90

$\bar{A} = 6$, $\bar{B} = 54$, $n = 5$, $\sigma_A = 2.83$,
 $\sigma_B = 24.17$, $\sum a_i b_i = 1960$.

Compute

$$\sum a_i b_i - n \bar{A} \bar{B} = 1960 - 5(6)(54) = 340$$
$$r = \frac{340}{5(2.83)(24.17)} = \mathbf{0.995}$$

$$\text{Covariance} = \frac{340}{5} = \mathbf{68.0}.$$

Caution

Correlation $\neq$ causation. Also r captures only *linear* relationships — a perfect parabola can have $r \approx 0$.

Data Reduction — Strategies Overview

Goal

Obtain a **reduced representation** that is much smaller yet produces (almost) the same analytical results.

Dimensionality reduction

PCA, wavelets,
attribute selection

Numerosity reduction

histograms, clustering,
sampling, regression

Data compression

lossless /
lossy encodings

Data cube aggregation summarise at a coarser level

Fewer columns

Dimensionality reduction removes or combines *attributes*.

Fewer rows / smaller values

Numerosity reduction replaces data by a smaller *representation* (a model, a sample, or a summary).

Attribute Subset Selection

Idea

Remove **irrelevant** or **redundant** attributes: fewer columns, faster training, better interpretability, same result quality.

Why not try every subset?

With n attributes there are 2^n subsets — for $n = 20$, 1,048,576. Exhaustive search is infeasible $\Rightarrow$ **greedy** methods.

Greedy methods

- **Forward selection** — start empty, add the best each round.
- **Backward elimination** — start full, drop the worst each round.
- **Combination** of the two.
- **Decision-tree induction** — attributes absent from the tree are deemed irrelevant.

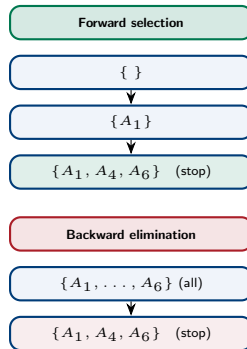


Figure: Both greedy searches converge to the same reduced set here.

Numerosity Reduction — Histograms and Clustering

Histograms

Partition values into **buckets** and store only the bucket summary (count/average).

- **Equal-width**: equal value ranges.
- **Equal-frequency**: equal counts per bucket.

Equal-width example

Data 4, 8, 15, 21, 21, 24, 25, 28, 34; range 4–34, three buckets of width $\frac{34-4}{3} = 10$:

- [4, 14): {4, 8} — count 2
- [14, 24): {15, 21, 21} — count 3
- [24, 34]: {24, 25, 28, 34} — count 4

Clustering as reduction

Replace the data by **cluster representatives** (centroids) plus cluster diameters — effective when data is naturally grouped.

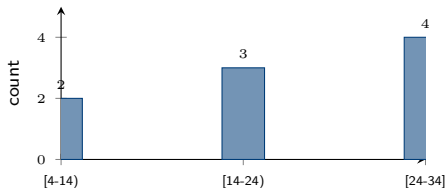


Figure: Nine values summarised by three bucket counts.

Numerosity Reduction — Sampling and Data Cube Aggregation

Sampling methods

- **SRSWOR** — simple random sample *without* replacement.
- **SRSWR** — *with* replacement.
- **Cluster sample** — sample whole groups (e.g. pages of tuples).
- **Stratified sample** — sample within each stratum, preserving proportions. **Best for skewed data.**

Why stratify

If 1% of customers are “premium”, a plain 5% random sample may contain almost none. Stratified sampling guarantees each group is represented.

Data cube aggregation

Summarise at a **coarser granularity**: replace daily sales by *quarterly* or *annual* totals.

Numeric illustration

Quarterly sales for a year:

Quarter	Q1	Q2	Q3	Q4	Year
Sales (Rs. lakh)	22	31	27	40	120

Four rows collapse to **one** — far less data, and often exactly the level at which the analysis is needed.

Data Transformation — Strategies

Standard strategies (Han & Kamber)

- 1 **Smoothing** — remove noise (binning, regression, clustering).
- 2 **Attribute construction** — build new features from existing ones.
- 3 **Aggregation** — summarise (daily → monthly).
- 4 **Normalization** — scale into a small, common range.
- 5 **Discretization** — numeric → interval/concept labels.
- 6 **Concept hierarchy generation** — e.g. *street* → *city* → *state*.

Attribute construction — an example

From height and weight construct

$$\text{BMI} = \frac{\text{weight (kg)}}{\text{height (m)}^2}$$

One well-chosen constructed feature can outperform many raw ones.

Why normalization matters

Distance-based methods (k -NN, k -means, SVM) and gradient descent are dominated by **large-scale** attributes. Income (in lakhs) would swamp age (in years) unless both are rescaled.

Normalization — The Three Standard Methods

1. Min-max normalization

Maps to a new range [new_min, new_max]:

$$v' = \frac{v - \min_A}{\max_A - \min_A}(\text{new_max} - \text{new_min}) + \text{new_min}$$

Preserves relationships exactly; **sensitive to outliers** and breaks if future data falls outside the original range.

2. Z-score (zero-mean) normalization

$$v' = \frac{v - \bar{A}}{\sigma_A}$$

Useful when min/max are unknown or outliers dominate.

3. Normalization by decimal scaling

$$v' = \frac{v}{10^j}$$

where j is the smallest integer such that $\max(|v'|) < 1$.

When to use which

- Bounded, outlier-free data → **min-max**.
- Unknown range / outliers → **z-score**.
- Quick, order-of-magnitude scaling → **decimal scaling**.

All three are computed on the *training* set and then *applied* to validation/test data.

Numerical Example — Normalizing an Income Value

Given

For attribute *income*: $\min_A = \text{Rs. } 12,000$, $\max_A = \text{Rs. } 98,000$, $\bar{A} = \text{Rs. } 54,000$, $\sigma_A = \text{Rs. } 16,000$. Normalize the value $v = \text{Rs. } 73,600$.

Min-max, to range $[0, 1]$

$$\begin{aligned} v' &= \frac{73600 - 12000}{98000 - 12000} (1 - 0) + 0 \\ &= \frac{61600}{86000} = \mathbf{0.716} \end{aligned}$$

Z-score

$$v' = \frac{73600 - 54000}{16000} = \frac{19600}{16000} = \mathbf{1.225}$$

The value lies 1.225 standard deviations **above** the mean.

Decimal scaling

Suppose the attribute ranges over -986 to 917 , so $\max |v| = 986$ and we need $j = 3$:

$$986 \rightarrow \frac{986}{1000} = \mathbf{0.986}, \quad -986 \rightarrow \mathbf{-0.986}$$

Common exam trap

Compute $\min / \max / \bar{A} / \sigma_A$ from the **training set only**. Using test-set statistics is a form of **data leakage**.

Discretization — Converting Numeric to Categorical

Why discretize?

- Some algorithms need **categorical** inputs (e.g. certain decision trees, association-rule mining).
- Reduces data size and noise sensitivity.
- Produces **human-readable** intervals: “young / middle-aged / senior”.

Two directions

Top-down (splitting): start with the whole range and split.

Bottom-up (merging): start with all points and merge neighbours (e.g. ChiMerge).

Techniques covered

- ➊ **Binning** — equal-width or equal-frequency (*unsupervised*).
- ➋ **Histogram analysis** — recursively apply to each bucket (*unsupervised*).
- ➌ **Clustering** — each cluster becomes an interval.
- ➍ **Decision-tree analysis** — splits chosen to maximise class purity (*supervised*).
- ➎ **Correlation analysis** — ChiMerge merges adjacent intervals with similar class distributions (*supervised*).

Numerical Example — Discretization by Binning

Ages of 12 students

18, 19, 19, 20, 21, 22, 22, 23, 25, 27, 30, 35

Equal-width binning ($k = 3$)

$$\text{Width} = \frac{35 - 18}{3} = \frac{17}{3} \approx 5.67$$

- $[18, 23.67)$: 18, 19, 19, 20, 21, 22, 22, 23 — **8 values**
- $[23.67, 29.33)$: 25, 27 — **2 values**
- $[29.33, 35]$: 30, 35 — **2 values**

Note the *uneven counts* — typical when data is skewed.

Equal-frequency binning ($k = 3$, 4 each)

- Bin 1: 18, 19, 19, 20 — label “**young**”
- Bin 2: 21, 22, 22, 23 — label “**mid**”
- Bin 3: 25, 27, 30, 35 — label “**senior**”

Counts are balanced, but the *widths* differ (3, 3, 11 years).

Trade-off

Equal-width keeps intervals interpretable;
equal-frequency keeps each bin statistically meaningful.
Choose by purpose.

Discretization by Clustering, Decision Trees and Correlation

By clustering

Apply e.g. k -means to the single attribute; each **cluster** becomes one interval. Respects the natural *gaps* in the data, unlike fixed-width bins. Can be applied recursively for a hierarchy.

By decision tree (supervised)

Use the **class labels**: choose split points that maximise information gain (minimise entropy):

$$\text{Entropy}(S) = - \sum_k p_k \log_2 p_k$$

Splits are placed where they best **separate the classes** — usually far better than unsupervised binning for classification.

By correlation analysis (ChiMerge)

Bottom-up:

- ① Treat each distinct value as its own interval.
- ② Compute χ^2 for every pair of *adjacent* intervals.
- ③ **Merge** the pair with the *lowest* χ^2 (most similar class distribution).
- ④ Repeat until a stopping criterion (max intervals or χ^2 threshold).

Rationale

Low χ^2 between neighbours means the class distribution does *not* change across the boundary — so the boundary is not worth keeping.

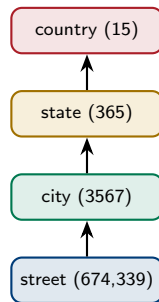
Concept Hierarchy Generation for Nominal Data

Four methods

- 1 **Explicit ordering by experts** at schema level:
street < city < state < country.
- 2 **Explicit data grouping** for small intermediate levels (e.g. group specific states into a region).
- 3 **Partial specification**, with the system completing the rest.
- 4 **Automatic generation** from the number of *distinct values*.

The heuristic behind (4)

The attribute with the **most distinct values** is placed at the *lowest* level of the hierarchy.



Ordered automatically
by the count of
distinct values:
fewer distinct values
⇒ higher level
(more general)

Figure: A concept hierarchy generated from distinct-value counts.

Quick Check — Round 3 (Data Preparation)

[Q1]

Name the four major tasks of data preprocessing.

[Q2]

For income with $\min = 10,000$, $\max = 90,000$, normalize $v = 50,000$ to $[0, 1]$ by min-max.

[Q3]

Data: 5, 10, 11, 13, 15, 35, 50, 55, 72. Using equal-frequency bins of size 3, smooth by *bin means*.

[Q4]

A χ^2 test on a 2×2 table gives $\chi^2 = 0.9$ with critical value 3.84 at $\alpha = 0.05$. What do you conclude?

Answers — Round 3

[A1]

Data cleaning, data integration, data reduction, data transformation (with discretization usually grouped under transformation).

[A2]

$$v' = \frac{50000 - 10000}{90000 - 10000} = \frac{40000}{80000} = \mathbf{0.5}.$$

[A3]

Bins: $\{5, 10, 11\}$, $\{13, 15, 35\}$, $\{50, 55, 72\}$.

Means: $\frac{26}{3} \approx \mathbf{8.67}$, $\frac{63}{3} = \mathbf{21}$, $\frac{177}{3} = \mathbf{59}$.

Smoothed: 8.67, 8.67, 8.67, 21, 21, 21, 59, 59, 59.

[A4]

Since $0.9 < 3.84$, we **fail to reject** the null hypothesis — there is **no significant evidence of correlation**; the attributes appear independent.

Summary of Unit 1

What we covered

- ① **What ML is** — Mitchell's $\langle T, P, E \rangle$; learning rules from data instead of writing them.
- ② **Problem framing** — goal, label, metric, features, baseline; good vs. hard ML problems; leakage.
- ③ **Applications** — image, speech, medical diagnosis, statistical arbitrage, association learning.
- ④ **Standard tasks** — classification, regression, ranking, clustering, dimensionality reduction; the ML pipeline.
- ⑤ **Learning stages** — features, labels, hyperparameters, train/validation/test, loss functions, hypothesis sets.
- ⑥ **Scenarios** — supervised, unsupervised, semi-supervised, transductive, on-line, reinforcement, active.
- ⑦ **Generalization** — overfitting/underfitting, bias–variance, remedies.
- ⑧ **Data preparation** — cleaning, integration, reduction, transformation and discretization, with calculations.

Good data and a well-framed question beat a fancy algorithm.

Practice Questions (Exam Oriented)

Short answer (2–5 marks)

- 1 State Mitchell's definition and illustrate $\langle T, P, E \rangle$ for two systems.
- 2 Differentiate supervised, unsupervised and reinforcement learning.
- 3 Distinguish parameters from hyperparameters with examples.
- 4 Why is accuracy misleading for imbalanced data? Suggest better metrics.
- 5 List the six data-quality dimensions.
- 6 Compare equal-width and equal-frequency binning.

Long answer (8–10 marks) — include numerics

- 7 Draw and explain the ML pipeline; state where most project effort goes and why.
- 8 Explain overfitting and underfitting with a diagram; give five remedies.
- 9 Given a contingency table, compute χ^2 and decide whether the attributes are correlated.
- 10 Given data values with min, max, mean and σ , apply all three normalization methods.
- 11 Given nine sorted values, perform smoothing by bin means and bin boundaries.
- 12 Explain the learning scenarios with one real application each.

Think & Explore (Take-Home)

Framing exercise

Pick *one* campus problem (library demand, mess wastage, dropout risk, placement prediction) and write a one-page **ML problem statement**:

- Goal, ideal outcome, label, metric.
- Features available *at prediction time*.
- A non-ML baseline to beat.
- One way the model could cause harm.

Coding (self-learning)

Load a public dataset (Iris, Titanic, or any CSV) with `pandas`. Report missing values, impute them, normalize one column three ways, and discretize one column into three bins.

Reading

- Mitchell, *Machine Learning* — Ch. 1.
- Mohri, Rostamizadeh & Talwalkar, *Foundations of ML* — Ch. 1.
- Han, Kamber & Pei, *Data Mining* — Ch. 2–3 (all of Part C).
- Google *Machine Learning Problem Framing* guide.

Reminder

Post your doubts on the course page; bring your framing exercise to the next tutorial for discussion.

References

Textbooks

- ① T. M. Mitchell, *Machine Learning*, McGraw Hill, 1997. **[Definitions, Ch. 1]**
- ② M. Mohri, A. Rostamizadeh, A. Talwalkar, *Foundations of Machine Learning*, 2nd ed., MIT Press, 2018. **[Stages & scenarios]**
- ③ J. Han, M. Kamber, J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed., Morgan Kaufmann. **[Part C: data preprocessing]**
- ④ E. Alpaydın, *Introduction to Machine Learning*, MIT Press.
- ⑤ C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer.
- ⑥ A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly.

Online

Google Developers — *Machine Learning Crash Course* and *Introduction to ML Problem Framing*; NPTEL — *Introduction to Machine Learning*, IIT Madras.

Course page: <https://www.gcjana.in/courses/shardauniversity/2601/machine-learning/>

Thank You

Any Questions?

“A breakthrough in machine learning would be worth ten Microsofts.”

— Bill Gates

Next: **Unit 2** — supervised learning algorithms in depth.

Post your doubts on the course page →

<https://www.gcjana.in/courses/shardauniversity/2601/machine-learning/#Post-doubts>